ปลายภาค 1/69 · การเขียนโปรแกรมเชิงวัตถุ (Java)

OOP สำหรับคนที่ยังไม่เข้าใจ เริ่มจากศูนย์จนอ่านโค้ดสอบได้

ทุกเรื่องเริ่มจากเรื่องเปรียบเทียบในชีวิตจริง แล้วค่อยไปดูโค้ด
และอธิบายโค้ดทีละบรรทัดว่าแต่ละส่วนทำหน้าที่อะไร

0 พื้นฐานที่ต้องรู้ก่อน | 7 Array | 8 Inheritance | 9 Abstraction | 10 Polymorphism | 11 Exception | 12 GUI

0 · พื้นฐาน

Class กับ Object ต่างกันอย่างไร



Class = แบบแปลนบ้าน
กำหนดว่าบ้านทุกหลังต้องมีอะไร (มีกี่ห้อง สีอะไร) และทำอะไรได้ (เปิดประตู เปิดไฟ) แต่แบบแปลนยังอยู่อาศัยไม่ได้



Object = บ้านที่สร้างเสร็จจริง
สร้างจากแบบเดียวกันได้หลายหลัง แต่ละหลังมีค่าของตัวเอง เช่น หลังหนึ่งสีฟ้า อีกหลังสีชมพู



`new` = สั่งช่างสร้างบ้าน 1 หลังจากแบบ
ทุกครั้งที่เขียน `new` จะได้ object ใหม่เพิ่ม 1 ตัว

```
// แบบแปลน (class)
class Dog {
  String name;            // ข้อมูล: หมามีชื่อ
  void bark() {           // ความสามารถ: เห่าได้
    System.out.println(name + " โฮ่ง!");
  }
}

// สร้างของจริง (object) 2 ตัว
Dog a = new Dog();  a.name = "ถุงเงิน";
Dog b = new Dog();  b.name = "ถุงทอง";
a.bark();   // ถุงเงิน โฮ่ง!
b.bark();   // ถุงทอง โฮ่ง!
```

จุด `.` อ่านว่า **“ของ”** เช่น `a.name` คือชื่อ**ของ** a และ `a.bark()` คือสั่งให้ a เห่า

ส่วนประกอบของ class มี 3 อย่างเสมอ

```java
public class Student {
  private String name;        // ①
  private double gpa;         // ①
  public Student(String name, double gpa) { // ②
    this.name = name;
    this.gpa  = gpa;
  }
  public boolean isHonor() {   // ③
    return gpa >= 3.50;
  }
  public void showInfo() {     // ③
    System.out.println(name + " | GPA: " + gpa);
  }
}

Student s = new Student("สมชาย", 3.25); // เรียก ②
s.showInfo();      // สมชาย | GPA: 3.25
s.isHonor();       // false
```

1. **Attribute (ข้อมูล)** สิ่งที่ object “มี” เขียนเป็น `ชนิด ชื่อ;` และมักเป็น `private` (ซ่อนไว้ ห้ามคนนอกแก้)
2. **Constructor (ตัวสร้าง)** ชื่อ**เหมือนชื่อ class** และ**ไม่มี return type** ทำงานครั้งเดียวตอนสั่ง `new` เพื่อใส่ค่าเริ่มต้น
3. **Method (ความสามารถ)** สิ่งที่ object “ทำได้” โดยคำหน้าชื่อคือชนิดของค่าที่ส่งกลับ

คำ	ความหมาย
`this.name`	name **ของ object ตัวนี้** (ไว้แยกจากพารามิเตอร์ที่ชื่อซ้ำ)
`void`	ทำงานอย่างเดียว ไม่ส่งค่ากลับ
`boolean` / `double` …	ส่งค่าชนิดนั้นกลับด้วย `return`
`public` / `private`	ใครเรียกก็ได้ / เรียกได้แค่ใน class เดียวกัน
`static`	เป็นของ class ไม่ใช่ของ object เช่น `main`

ตัวแปร object คือ “รีโมท” ไม่ใช่ตัวของจริง



ตัวแปรชนิดพื้นฐาน (`int` , `double` , `boolean`) เก็บ**ค่าไว้ในกล่องโดยตรง**



ตัวแปร object (`String` , `Dog` , `Student`) เก็บแค่**รีโมท** ซึ่งชี้ไปหาของจริงที่อยู่ที่อื่นในหน่วยความจำ



`null` **= รีโมทที่ยังไม่ได้จับคู่กับทีวีเครื่องไหน** ถ้ากดปุ่มบนรีโมทนี้ จะได้ **NullPointerException**

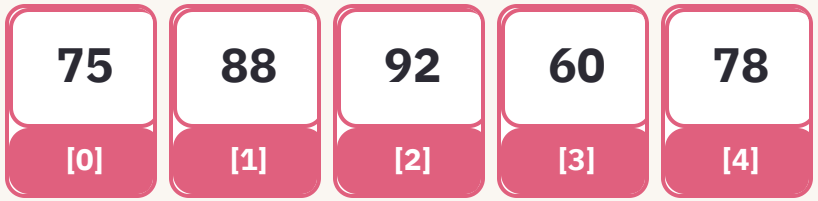
```java
int x = 5;                    // กล่อง x มีเลข 5 อยู่ข้างใน
Dog d;                        // มีรีโมทแล้ว แต่ยังไม่มีหมา
d = new Dog();                // สร้างหมา แล้วจับคู่กับรีโมท d
Dog e = d;                    // รีโมทอีกอัน ชี้หมาตัวเดียวกัน
e.name = "โบ้";
System.out.println(d.name); // โบ้  (เพราะเป็นหมาตัวเดียวกัน)

Dog z = null;
z.bark();                     // ✗ NullPointerException
```

เรื่องนี้เป็นพื้นฐานของบทที่ยากที่สุด (บท 10) ชนิดของรีโมทกำหนดว่า**มีปุ่มอะไรให้กด** ส่วนของจริงที่ถูกชี้กำหนดว่า**กดแล้วเกิดอะไรขึ้น**

Array = ตู้ล็อกเกอร์ที่ช่องแรกคือเลข 0

ถ้า `int[] scores = {75, 88, 92, 60, 78};` ตู้จะหน้าตาแบบนี้



- มี 5 ช่อง ช่องแรกคือ `[0]` และช่องสุดท้ายคือ `[4]` (**ไม่ใช่** [5])
- `scores.length` = 5 คือจำนวนช่อง (เขียน**ไม่มีวงเล็บ**)
- ตู้สร้างแล้ว**ขยายขนาดไม่ได้**
- เปิดช่อง [5] ซึ่งไม่มีอยู่จริง จะได้ **ArrayIndexOutOfBoundsException**
- ตู้ใหม่ที่ยังไม่ใส่อะไร: ถ้าเป็น `int` ทุกช่องเป็น 0 ถ้าเป็น object ทุกช่องเป็น `null`

```
int sum = 0;
for (int i = 0; i < scores.length; i++) {
  sum = sum + scores[i];
}
```

ลองไล่ลูปทีละรอบ (โจทย์ทายผลจะเป็นแบบนี้)

รอบ	i	scores[i]	sum หลังบวก
1	0	75	75
2	1	88	163
3	2	92	255
4	3	60	315
5	4	78	**393**
—	5	5 < 5 เป็นเท็จ จึงหยุดลูป	

บท 7 · Array 2 มิติ

Array 2 มิติ = ผังที่นั่งโรงหนัง [แถว][ที่นั่ง]

```
int[][] s = { {75, 80, 70},    // แถว 0
              {88, 70, 95},    // แถว 1
              {92, 85, 88} };  // แถว 2
```

	คอลัมน์ 0	คอลัมน์ 1	คอลัมน์ 2
แถว 0	75	80	70
แถว 1	88	70	**95**
แถว 2	92	85	88

`s[1][2]` = แถว 1 คอลัมน์ 2 = **95** · `s.length` = 3 แถว · `s[0].length` = 3 คอลัมน์

Array ของ object = ตู้ที่ในแต่ละช่องเก็บรีโมท

```
Student[] st = new Student[3];
// ได้ตู้ 3 ช่อง แต่ทุกช่องยังเป็น null (ยังไม่มีนักเรียน!)

st[0] = new Student("สมชาย", 3.25); // ต้อง new ทีละคน
st[0].showInfo();   // ✓
st[1].showInfo();   // × NullPointerException
```

ต้องจำ: `new Student[3]` สร้างแค่**ตู้** ยังไม่ได้สร้างนักเรียน ต้อง `new Student(...)` ใส่ทีละช่องเอง

บท 7 · ArrayList

ArrayList = แถวคิวที่ยืดหดได้เอง



มีคนมาต่อแถว คิวก็ยาวขึ้น มีคนออกจากแถว คนข้างหลังก็**ขยับขึ้นมาแทนเอง** ต่างจาก Array ที่ขนาดตายตัว

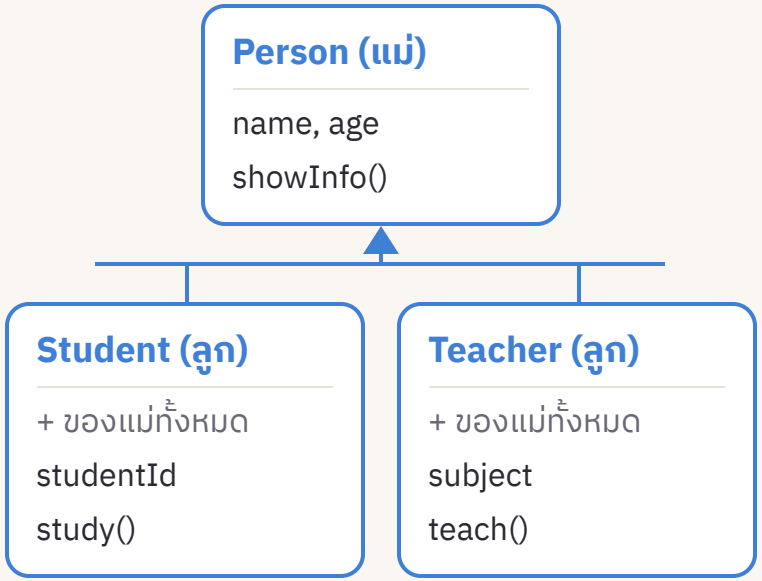
```
ArrayList<String> q = new ArrayList<>();
q.add("A");        // [A]
q.add("B");        // [A, B]
q.add("C");        // [A, B, C]
q.add(1, "X");     // แทรกที่ช่อง 1 → [A, X, B, C]
q.get(2);          // "B"
q.remove(0);       // [X, B, C]  ทุกคนขยับขึ้น
q.size();          // 3
```

คำสั่ง	ความหมายภาษาคน
`add(x)`	ต่อท้ายแถว
`add(i, x)`	แทรกเข้าไปที่ตำแหน่ง i
`get(i)`	ดูว่าตำแหน่ง i คือใคร
`set(i, x)`	เปลี่ยนคนที่ตำแหน่ง i
`remove(i)`	เอาคนที่ตำแหน่ง i ออก
`size()`	ตอนนี้มีกี่คน
`contains(x)`	มี x อยู่ในแถวไหม
`isEmpty()` / `clear()`	แถวว่างไหม / ไล่ทุกคนออก

`ArrayList<int>` **ผิด** ต้องเขียน `<Integer>` หรือ `<Double>` เพราะในวงเล็บแหลมต้องเป็นชื่อ class (ตัวพิมพ์ใหญ่)

Inheritance = ลูกได้รับมรดกจากพ่อแม่

ทุกอย่างที่แม่มี ลูกได้ไปด้วยอัตโนมัติ ไม่ต้องเขียนซ้ำ แล้วลูกยัง **เพิ่มของของตัวเอง**ได้




```
class Student extends Person {   // extends = "เป็นลูกของ"
  private String studentId;
  ...
}
```

ทดสอบว่าควรใช้ extends ไหม

พูดประโยค **"A เป็น B ชนิดหนึ่ง"** แล้วดูว่าฟังดูถูกไหม

- Student **เป็น** Person ชนิดหนึ่ง ✓ → ใช้ `extends` (**IS-A**)
- Car **เป็น** Engine ชนิดหนึ่ง × แต่ Car **มี** Engine → ให้ Engine เป็น attribute ของ Car (**HAS-A**)

ลูก**มีแม่ได้คนเดียว** (extends ได้ class เดียว) · ลูกใช้ของที่แม่ตั้งเป็น `private` โดยตรงไม่ได้ แต่ใช้ของที่เป็น `protected` และ `public` ได้

บท 8 · super

super = เรียกใช้ของพ่อแม่ และต้องสร้างพ่อแม่ก่อนเสมอ

```
class Person {
  String name;
  Person(String name) {
    this.name = name;
    System.out.println("① สร้างส่วน Person");
  }
}
class Student extends Person {
  String id;
  Student(String name, String id) {
    super(name);        // ส่ง name ไปให้ constructor แม่
    this.id = id;
    System.out.println("② สร้างส่วน Student");
  }
}
new Student("Alice", "65001");
// ① สร้างส่วน Person
// ② สร้างส่วน Student
```

🏗️ สร้างบ้านต้อง**ลงฐานราก (แม่) ก่อน** แล้วค่อยสร้างชั้นบน (ลูก) ผลลัพธ์จึงขึ้นของแม่ก่อนเสมอ

เขียน	หมายถึง
`super(name)`	เรียก constructor ของแม่ ต้องอยู่**บรรทัดแรก**ของ constructor ลูก
`super.showInfo()`	เรียกเมธอดเวอร์ชันของแม่
`this.id`	attribute ของตัวเอง

ถ้าไม่ได้เขียน `super(...)` เลย Java จะแอบใส่ `super()` (แบบไม่มีพารามิเตอร์) ให้ ถ้าแม่**ไม่มี** constructor แบบนั้น จะ **compile error**

บท 8 · Override

Override = ลูกทำสิ่งเดียวกันในแบบของตัวเอง

🗣️ แม่สอนว่า “ทักทาย” คือพูดว่า *สวัสดี* แต่ลูกที่ไปเรียนต่างประเทศ ทักทายด้วย *Hello* แทน ชื่อการกระทำเหมือนเดิม แต่**วิธีทำ**เปลี่ยนไป

```
class Animal {
  void makeSound() { System.out.println("..."); }
}
class Dog extends Animal {
  @Override                              // บอกว่าตั้งใจเขียนทับ
  void makeSound() { System.out.println("โฮ่ง"); }
}
new Dog().makeSound();    // โฮ่ง (ใช้ของลูก)
```

เงื่อนไขของ Override

- ชื่อเมธอดและวงเล็บพารามิเตอร์**ต้องเหมือนแม่ทุกตัวอักษร**
- ระดับการเข้าถึงต้องเท่าเดิมหรือ**กว้างกว่า** เช่น แม่เป็น public ลูกจะเป็น private ไม่ได้
- เมธอด `private`, `final`, `static` และ constructor **override ไม่ได้**

Overriding (บท 8, 10)	Overloading (ชื่อคล้าย อย่าสับสน)
อยู่ใน**แม่กับลูก**	อยู่ใน**class เดียวกัน**
พารามิเตอร์**เหมือน**กัน	พารามิเตอร์**ต่าง**กัน เช่น add(int,int) กับ add(double,double)
Java เลือกตอน**รัน**	Java เลือกตอน **compile**

บท 9 · Abstract class

Abstract class = แม่พิมพ์ที่ยังไม่เสร็จ

🔷 ถ้ามีคนบอกว่า “วาด**รูปทรง**มาหนึ่งรูป” คุณจะถามกลับว่ารูปทรงอะไร เพราะ “รูปทรง” เป็นแค่**แนวคิด** ส่วนวงกลมหรือสี่เหลี่ยมถึงจะวาดได้จริง

- รูปทรงทุกแบบ**ต้อง**หาพื้นที่ได้ แต่สูตรต่างกัน แม่จึงประกาศไว้แค่ชื่อเมธอด (**abstract method** ซึ่งไม่มีเนื้อ ไม่มี {})
- ลูกทุกตัว**ถูกบังคับ**ให้เขียนเนื้อเมธอดนั้น
- สั่ง `new Shape()` **ไม่ได้** เพราะแม่พิมพ์ยังไม่สมบูรณ์

```
abstract class Shape {
  abstract double calculateArea();  // ไม่มี { } ปิดด้วย ;
}
class Circle extends Shape {
  double radius;
  Circle(double r) { radius = r; }
  @Override
  double calculateArea() { return Math.PI * radius * radius; }
}
class Rectangle extends Shape {
  double w, h;
  Rectangle(double w, double h) { this.w = w; this.h = h; }
  @Override
  double calculateArea() { return w * h; }
}
Shape s1 = new Shape();      // ✗ ห้าม
Shape s2 = new Circle(5);    // ✓ รีโมทแม่ ชี้ของจริงที่เป็นลูก
```

ใน UML ชื่อ class ที่เป็น**ตัวเอียง** หรือมีคำว่า **«abstract»** กำกับ แปลว่าต้องเขียน `abstract`

บท 9 · Interface

Interface = สัญญาที่บอกว่า “ต้องทำอะไรได้บ้าง”

🔌 เต้าเสียบไฟกำหนดแค่**รูปทรงหัวปลั๊ก** ไม่สนใจว่าเครื่องที่เสียบจะเป็นพัดลมหรือทีวี ขอแค่หัวปลั๊กตรงตามรูป ก็เสียบได้

```
interface PaymentMethod {          // สัญญา
  void pay(double amount);         // ต้องจ่ายเงินได้
}
class CashPayment implements PaymentMethod {
  public void pay(double amount) {    // ต้องมี public
    System.out.println("จ่ายเงินสด " + amount);
  }
}
class QRPayment implements PaymentMethod {
  public void pay(double amount) {
    System.out.println("สแกนจ่าย " + amount);
  }
}
```

	abstract class	interface
เปรียบเหมือน	แม่พิมพ์ที่ยังไม่เสร็จ	สัญญา / ใบรับรอง
ลูกใช้คำว่า	`extends`	`implements`
ใช้ได้กี่อัน	แค่ **1**	**หลายอัน** คั่นด้วย ,
มีข้อมูล (attribute)?	มีได้	มีได้แค่ค่าคงที่
มี constructor?	มี	ไม่มี
สั่ง new ได้?	ไม่ได้	ไม่ได้

```
class Phone implements Camera, MusicPlayer {}  // ได้หลายอัน
class FullTime extends Employee implements Taxable {}
// extends ต้องมาก่อน implements
```

เมธอดที่ implement จาก interface ต้องเขียน `public` เสมอ ถ้าลืมจะ compile error

บท 10 · Polymorphism

Polymorphism = สั่งคำเดียว แต่ละตัวทำไม่เหมือนกัน

📢 ครูตะโกนว่า “**ทุกตัวส่งเสียง!**” หมาเห่า แมวร้องเหมียว วัวร้องมอ ครูไม่ต้องรู้ว่าใครเป็นสัตว์อะไร แค่สั่งคำเดียว

```
Animal[] zoo = { new Dog(), new Cat(), new Cow() };
for (Animal a : zoo) {
  a.makeSound();      // คำสั่งเดียวกันทุกตัว
}
// Dog barks: Woof!
// Cat meows: Meow!
// Cow moos: Moo!
```

ถ้าเพิ่มเป็ดเข้ามา แค่สร้าง `class Duck extends Animal` แล้วใส่ลง array **โดยไม่ต้องแก้ลูปเลย**

กฎรีโมท (ย้อนไปดูหน้า 4)

```
Animal a = new Dog();
//  ↑ รีโมทแบบ Animal    ↑ ของจริงเป็น Dog
```

- **กดปุ่มอะไรได้** ดูจากรีโมท (Animal) ถ้า Animal ไม่มีปุ่ม `guardHouse()` ก็ **compile error** ถึงแม้หมาจะเฝ้าบ้านได้
- **กดแล้วเกิดอะไร** ดูจากของจริง (Dog) `a.makeSound()` จึงได้เสียงหมา

ข้อยกเว้นที่ชอบออกสอบ: attribute (ตัวแปรใน class) **ไม่ใช้กฎนี้** ถ้าแม่กับลูกมี `name` ทั้งคู่ `a.name` จะได้ค่าของ**รีโมท (แม่)**

บท 10 · Casting

เปลี่ยนรีโมท: Upcasting และ Downcasting

	Upcasting ⬆️	Downcasting ⬇️
ทิศทาง	ลูก → แม่	แม่ → ลูก
เขียนยังไง	`Animal a = dog;` ทำให้อัตโนมัติ	`Dog d = (Dog) a;` ต้องเขียน (ชนิด) เอง
ปลอดภัย?	ปลอดภัยเสมอ	ถ้าของจริงไม่ใช่ชนิดนั้น จะพังตอนรัน

🐱 เอารีโมท “สัตว์” ที่จริง ๆ ชี้ไปหา**แมว** แล้วไปบังคับว่า “นี่คือหมา” Java ยอมให้ compile ผ่าน แต่ตอนรันจะเจอ **ClassCastException**

```
Animal a = new Dog();
Dog d = (Dog) a;          // ✓ ของจริงเป็นหมา
d.guardHouse();           // ✓ ตอนนี้มีปุ่มนี้แล้ว

Animal x = new Cat();
Dog y = (Dog) x;          // ✗ ClassCastException

// วิธีปลอดภัย: เช็คด้วย instanceof ก่อนแปลง
if (x instanceof Dog) {
  Dog z = (Dog) x;
  z.guardHouse();
}
```

`x instanceof Dog` อ่านว่า “x เป็นหมาจริงไหม” ได้ผลเป็น `true` หรือ `false`

บท 8 + 10 · วิธีทำข้อสอบ

สูตร 4 ขั้นสำหรับข้อ “ผลลัพธ์ของโปรแกรมคือ?”

1. **เจอ `new` ให้ดูว่าเป็น class อะไร** ถ้าเป็นลูก ให้รัน constructor ของ **แม่ก่อน** แล้วค่อยรันของลูก
2. **เจอการเรียกเมธอด ให้ถามว่ารีโมทมีปุ่มนี้ไหม** ถ้าไม่มี ให้ตอบว่า compile error ทันที
3. **ถ้ามีปุ่ม ให้ไปดูของจริง** ถ้าของจริงเป็นลูกที่ override เมธอดนี้ไว้ ให้ใช้เวอร์ชันลูก ถ้าไม่ได้ override ให้ใช้ของแม่
4. **เจอ `super.xxx()` ข้างในเมธอดลูก** ให้รันเวอร์ชันของแม่ตรงนั้น แล้วกลับมาทำบรรทัดต่อไปของลูก

```
class A {
  A() { System.out.print("A "); }
  void hi() { System.out.print("hiA "); }
}
class B extends A {
  B() { System.out.print("B "); }
  void hi() { super.hi(); System.out.print("hiB "); }
}
A obj = new B();    // ขั้น 1 → A B
obj.hi();           // ขั้น 2: A มีปุ่ม hi ✓
                    // ขั้น 3: ของจริงคือ B → hi ของ B
                    // ขั้น 4: super.hi() → hiA แล้วต่อ hiB
```

คำตอบ

```
A B hiA hiB
```

Exception = ตาข่ายนิรภัยกันโปรแกรมพัง



นักกายกรรมเล่นบนเชือก (**try**) ถ้าตกลงมา ก็มีตาข่าย (**catch**) รองรับไว้ แล้วไม่ว่าจะตกหรือไม่ จบการแสดงก็ต้องโค้งคำนับเสมอ (**finally**)

```java
try {
  // โค้ดที่อาจจะพัง
  int r = 10 / 0;
  System.out.println("ไม่ถูกพิมพ์");
} catch (ArithmeticException e) {
  // ทำอะไรเมื่อพังแบบนี้
  System.out.println("หารศูนย์ไม่ได้");
} finally {
  System.out.println("จบ");        // ทำเสมอ
}
// หารศูนย์ไม่ได้
// จบ
```

try รันไปทีละบรรทัด → **พัง?** ข้ามบรรทัดที่เหลือใน try ทันที → **catch** เลือกตัว **แรก** ที่ชนิดตรงกัน → **finally** ทำเสมอ

ชื่อ Exception	เกิดเมื่อ
ArithmeticException	หาร int ด้วย 0
ArrayIndexOutOfBoundsException	เปิดช่อง array ที่ไม่มีอยู่
NumberFormatException	แปลงข้อความที่ไม่ใช่ตัวเลข เช่น `Integer.parseInt("ABC")`
NullPointerException	ใช้รีโมทที่เป็น null
InputMismatchException	ใช้ Scanner รอรับตัวเลข แต่ผู้ใช้พิมพ์ตัวอักษร
ClassCastException	downcast ผิดชนิด (บท 10)

ชื่อ Exception บอกสาเหตุอยู่แล้ว ถ้าอ่านคำแรกของชื่อออก ก็เดาได้ว่าเกิดจากอะไร

throw คือ “โยน” ส่วน throws คือ “ป้ายเตือน”

🚨 **throw** = เห็นข้อมูลผิดแล้วกดสัญญาณเตือนเอง ณ บรรทัดนั้น
throws = ป้ายหน้าห้องที่บอกว่า “ห้องนี้อาจมีสัญญาณดัง คนเรียกใช้เตรียมรับมือด้วย”

```
public void setAge(int age) {
  if (age < 0) {
    throw new IllegalArgumentException("Age cannot be negative");
  }
  this.age = age;
}

public void readFile() throws IOException { ... }
```

สร้าง Exception ของตัวเอง (โครงที่ตายตัว)

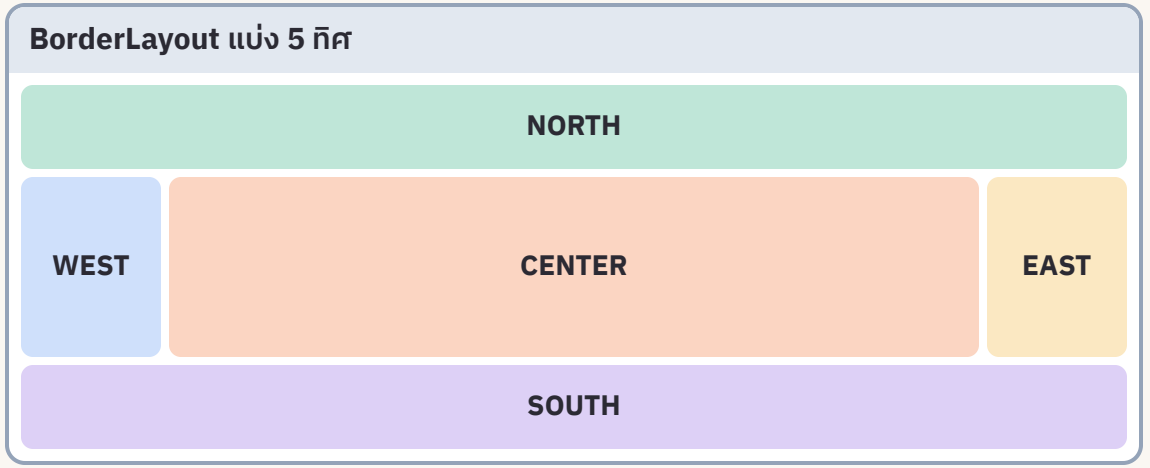
```
class InsufficientBalanceException extends Exception {
  InsufficientBalanceException(String msg) {
    super(msg);
  }
}
```

Unchecked	Checked
compiler **ไม่บังคับ**ให้ดัก	compiler **บังคับ**ให้ try-catch หรือเขียน throws
5 ตัวในหน้าที่แล้วทั้งหมด	FileNotFoundException, IOException (เรื่องไฟล์)

ถ้ามีหลาย catch ให้เรียง**ตัวเจาะจงไว้ก่อน** แล้ว `catch (Exception e)` ไว้**ท้ายสุด** ถ้าสลับกันจะ compile error

GUI = จัดร้านอาหาร

ในร้าน	ใน Java Swing
ตัวร้าน	**JFrame** หน้าต่างหลัก
โต๊ะแต่ละตัว	**JPanel** ใช้จัดกลุ่มของ
ป้ายเมนู	**JLabel** ข้อความที่ผู้ใช้แก้ไม่ได้
กระดาษให้ลูกค้าเขียน	**JTextField** 1 บรรทัด / **JTextArea** หลายบรรทัด
กริ่งเรียกพนักงาน	**JButton**
เลือกท็อปปิงได้หลายอย่าง	**JCheckBox**
เลือกขนาดได้อย่างเดียว	**JRadioButton** + ButtonGroup
รายการให้เลือก	**JComboBox** (dropdown)
ป๊อปอัปแจ้งลูกค้า	**JOptionPane**
วิธีจัดโต๊ะในร้าน	**Layout Manager**



- **FlowLayout** เรียงจากซ้ายไปขวาเหมือนตัวหนังสือ เต็มแล้วขึ้นบรรทัดใหม่
- **GridLayout(3, 2)** ตาราง 3 แถว 2 คอลัมน์ ทุกช่องขนาดเท่ากัน
- ฟอร์มจริงมักเอา JPanel ที่เป็น Grid ไปใส่ใน**ทิศ**ของ BorderLayout อีกที

โปรแกรม GUI ทุกตัวมีโครงเดียวกัน

```
import javax.swing.*;  import java.awt.*;
public class HelloGUI extends JFrame {     // ①
  private JTextField nameField;            // ②
  private JLabel resultLabel;

  public HelloGUI() {
    nameField = new JTextField(15);
    JButton btn = new JButton("ทักทาย");
    resultLabel = new JLabel("กรุณากรอกชื่อ");
    setLayout(new FlowLayout());           // ③
    add(nameField); add(btn); add(resultLabel);
    btn.addActionListener(e -> greet());   // ④
    setTitle("Hello"); setSize(400, 150);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  }
  private void greet() {                   // ⑤
    String name = nameField.getText().trim();
    if (name.isEmpty()) resultLabel.setText("กรุณากรอกชื่อ");
    else resultLabel.setText("สวัสดี " + name);
  }
  public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
        new HelloGUI().setVisible(true));    // ⑥
  }
}
```

1. หน้าต่างของเรา**เป็น JFrame ชนิดหนึ่ง** ซึ่งก็คือ inheritance จากบท 8
2. ประกาศช่องกรอกและป้ายไว้**ด้านบน** เพื่อให้เมธอดอื่นเรียกใช้ได้
3. เลือกวิธีจัดวางก่อน แล้วค่อย `add` ของลงหน้าต่าง
4. **ผูกปุ่มกับงาน** คือเมื่อคลิกปุ่ม ให้ไปทำ `greet()` ถ้าไม่มีบรรทัดนี้ ปุ่มจะกดแล้วไม่เกิดอะไร
5. `getText()` อ่านสิ่งที่ผู้ใช้พิมพ์ · `trim()` ตัดช่องว่างหัวท้าย · `setText()` เปลี่ยนข้อความที่แสดง
6. สร้างหน้าต่างแล้วสั่ง**แสดงผล** ถ้าลืม `setVisible(true)` จะไม่เห็นอะไรขึ้นมาเลย

บท 11 + 12 · ข้อเติมโค้ด

ช่องตัวเลขใน GUI ต้องกันผู้ใช้พิมพ์ผิดเสมอ

```
private void calculate() {
  try {
    double a = Double.parseDouble(firstField.getText().trim());
    double b = Double.parseDouble(secondField.getText().trim());
    resultLabel.setText("ผลลัพธ์ = " + (a + b));
  } catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this,
        "กรุณากรอกข้อมูลเป็นตัวเลข",
        "ข้อมูลไม่ถูกต้อง",
        JOptionPane.ERROR_MESSAGE);
  }
}
```

ทุกอย่างที่ได้จาก `getText()` เป็น**ข้อความ** (String) ถ้าจะเอาไปคำนวณ ต้องแปลงด้วย `parseDouble` หรือ `parseInt` ก่อน และถ้าผู้ใช้พิมพ์ “abc” จะเกิด **NumberFormatException**

กล่องป๊อปอัป JOptionPane	ใช้ทำอะไร / ได้อะไรกลับมา
`showMessageDialog`	แจ้งข้อความ แล้วมีปุ่ม OK
`showInputDialog`	ให้พิมพ์ ได้ String กลับมา หรือได้ **null** ถ้ากด Cancel
`showConfirmDialog`	ถาม Yes/No ได้ int กลับมา ให้เทียบกับ `YES_OPTION`

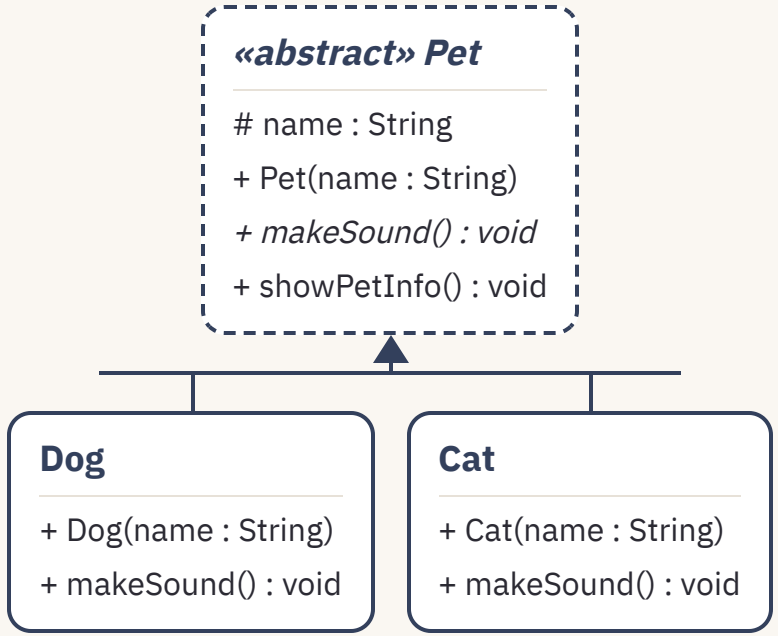
ไอคอนของป๊อปอัป

`INFORMATION_MESSAGE` ข้อมูล · `WARNING_MESSAGE` เตือน · `ERROR_MESSAGE` ผิดพลาด · `QUESTION_MESSAGE` คำถาม · `PLAIN_MESSAGE` ไม่มีไอคอน

ข้อเติมโค้ดมักเว้นว่างตรง: `try` · `catch` · `NumberFormatException` · `parseDouble` · `getText` · `setText` · `addActionListener` · `showMessageDialog` · `setVisible`

ตอนที่ 2.1 · UML → โค้ด

อ่าน UML Class Diagram ให้เป็นโค้ดทีละขั้น



สัญลักษณ์	แปลว่า
+ # −	public · protected · private
`name : String`	เขียนกลับด้านเป็น `String name;`
ตัวเอียง / «abstract»	ใส่คำว่า `abstract`
ลูกศรหัวสามเหลี่ยมเส้นทึบ / เส้นประ	`extends` / `implements`

```java
public abstract class Pet {
  protected String name;
  public Pet(String name) { this.name = name; }
  public abstract void makeSound();
  public void showPetInfo() {
    System.out.print(name + " Making Sound : ");
    makeSound();
  }
}
public class Dog extends Pet {
  public Dog(String name) { super(name); }
  @Override
  public void makeSound() {
    System.out.println("Bok Bok...");
  }
}
```

1. เขียนแม่ก่อน: attribute → constructor → เมธอด
2. ลูกทุกตัว: `extends` + constructor ที่มี `super(...)` บรรทัดแรก
3. ลูกเขียนเนื้อ abstract method ให้ครบ พร้อม `@Override`

ทวน

พจนานุกรมคำสำคัญ

เห็นคำนี้ในโค้ด ให้นึกถึงความหมายนี้

คำ	นึกถึง
`class`	แบบแปลน
`new`	สร้างของจริง 1 ชิ้น
`this`	ตัวฉันเอง
`extends`	เป็นลูกของ (IS-A)
`super`	พ่อแม่ของฉัน
`@Override`	ฉันทำเรื่องนี้ในแบบของฉัน
`abstract`	ยังไม่เสร็จ ลูกต้องทำต่อ
`interface` / `implements`	สัญญา / ลงชื่อรับสัญญา
`instanceof`	ของจริงเป็นชนิดนี้ไหม

คำ	นึกถึง
`private` / `protected` / `public`	เฉพาะฉัน / ครอบครัว / ทุกคน
`static`	ของส่วนกลาง (เป็นของ class)
`final`	ห้ามแก้ ห้าม override
`null`	รีโมทที่ไม่ได้ชี้อะไร
`try` / `catch` / `finally`	ลองทำ / ตาข่าย / ทำเสมอ
`throw` / `throws`	กดสัญญาณเตือน / ป้ายเตือน
`ArrayList<T>`	แถวคิวที่ยืดหดได้
`addActionListener`	ผูกปุ่มกับงาน
`getText` / `setText`	อ่าน / เขียนข้อความบนหน้าจอ

กลยุทธ์สำหรับมือใหม่

เก็บคะแนนให้ได้มากที่สุดแม้ยังไม่เข้าใจทั้งหมด

ตอน 1 ปรนัย

- ข้อที่ถามว่า “compile error ไหม” ให้มองหา `new` abstract หรือ interface, เรียกปุ่มที่รีโมทไม่มี, ลืม `super`, catch ผิดลำดับ
- ข้อ array: ช่องสุดท้ายคือ length−1
- ข้อ try: บรรทัดหลังจุดที่พังไม่ถูกรัน และ finally ทำเสมอ
- ไล่ค่าตัวแปรด้วยการจดลงกระดาษทุกครั้ง

ตอน 2.1 UML

- คะแนนให้ตามส่วน ถึงทำไม่ครบก็เขียน **โครง**ให้ครบไว้ก่อน
- เขียน `class X extends Y`, attribute และ constructor พร้อม `super` ให้ถูก ก็ได้คะแนนไปเกินครึ่ง
- main: ใช้ `ArrayList<แม่>` + `add(new ลูก(...))` + for-each เรียกเมธอด

ตอน 2.2 เติมโค้ด

- ดูคำที่อยู่**รอบ ๆ ช่องว่าง** ถ้าเห็นเปิดปีกกาตามด้วย `catch` ช่องข้างบนคือ `try`
- เห็น `parseInt` หรือ `parseDouble` ให้ catch `NumberFormatException`
- เห็นการหาร ให้ catch `ArithmeticException`
- เห็นปุ่ม ให้ใช้ `addActionListener(e -> ...)`
- บรรทัดสุดท้ายของ main มักเป็น `setVisible(true)`